EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Programmation en C

Algorithmique et Programmation :

Fonctions et pointeurs

MOHAMMED VI
POLYTECHNIC
UNIVERSITY

Introduction

Certains problèmes conduisent à des programmes longs, difficiles à écrire et à comprendre. On les découpe en des parties appelées **sous-programmes** ou **modules**

Les **fonctions** et les **procédures** sont des modules (groupe d'instructions) indépendants désignés par un nom. Elles ont plusieurs **intérêts** :

- permettent de **"factoriser" les programmes**, càd de mettre en commun les parties qui se répètent
- permettent une **structuration** et une **meilleure lisibilité** des programmes
- **facilitent la maintenance** du code (il suffit de modifier une seule fois)
- ces procédures et fonctions peuvent éventuellement être **réutilisées** dans d'autres programmes

Fonctions

Le **rôle** d'une fonction en programmation est similaire à celui d'une fonction en mathématique : elle **retourne un résultat à partir des valeurs des paramètres**

Une fonction s'écrit en dehors du programme principal sous la forme :

Fonction nom_fonction (paramètres et leurs types) : type_fonction

Instructions constituant le corps de la fonction
retourne …

FinFonction

Pour le choix d'un nom de fonction il faut respecter les mêmes règles que celles pour les noms de variables
type_fonction est le type du résultat retourné
L'instruction **retourne** sert à retourner la valeur du résultat

Fonctions : exemple

La fonction SommeCarre suivante calcule la somme des carrées de deux réels x et y :

```
Fonction SommeCarre (x : réel, y: réel ) : réel
        variable z : réel
        z ←x^2+y^2
        retourne (z)
FinFonction
```

La fonction Pair suivante détermine si un nombre est pair :

```
Fonction  Pair (n : entier ) : booléen
        retourne (n%2=0)
FinFonction
```

Utilisation des fonctions

L'utilisation d'une fonction se fera par simple écriture de son nom dans le programme principale. Le résultat étant une valeur, devra être affecté ou être utilisé dans une expression, une écriture, ...

Exepmle : **Algorithme exepmleAppelFonction**

```
variables    z : réel,  b : booléen
Début
    b ←Pair(3)
    z ←5*SommeCarre(7,2)+1
    écrire("SommeCarre(3,5)= ", SommeCarre(3,5))
Fin
```

Lors de l'appel Pair(3) le **paramètre formel** n est remplacé par le **paramètre effectif** 3

Procédures

Dans certains cas, on peut avoir besoin de répéter une tache dans plusieurs endroits du programme, mais que dans cette tache on ne calcule pas de résultats ou qu'on calcule plusieurs résultats à la fois

Dans ces cas on ne peut pas utiliser une fonction, on utilise une **procédure**

Une **procédure** est un sous-programme semblable à une fonction mais qui **ne retourne rien**

Une procédure s'écrit en dehors du programme principal sous la forme :

Procédure nom_procédure (paramètres et leurs types)

Instructions constituant le corps de la procédure

FinProcédure

Remarque : une procédure peut ne pas avoir de paramètres

Appel d'une procédure

L'appel d'une procédure, se fait dans le programme principale ou dans une autre procédure par une instruction indiquant le nom de la procédure :

```
Procédure exemple_proc (…)
        …
FinProcédure

Algorithme exepmleAppelProcédure
Début
        exemple_proc (…)
        …
Fin
```

Remarque : contrairement à l'appel d'une fonction, on ne peut pas affecter la procédure appelée ou l'utiliser dans une expression. L'appel d'une procédure est une instruction autonome.

Fonctions en C

Les principes

- En C, tout est fonction (Définition, Déclaration et Appel).
- Une fonction peut avoir autant de paramètres que l'on veut, même aucun
  comme void main(void))
- Une fonction renvoie une valeur ou aucune.
- Les variables déclarées dans une fonction sont locales à cette fonction
- Une fonction possède un et un seul point d'entrée, mais éventuellement plusieurs points de sortie (à l'aide du mot return).

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Déclarer une fonction

TYPE de la valeur de retour

3 doubles comme paramètres

```
int     print_table(double start, double end, double step)
{
        double      d;
        int         lines = 1;

        printf("Celsius\tFarenheit\n");
        for(d = start; d <= end; d += step, lines++)
                printf("%.1lf\t%.1lf\n", d, d * 1.8 + 32);

        return lines;
}
```

Nom de la fonction

Valeur renvoyée

Appeler une fonction

IMPORTANT: cette instruction spécifie comment la fonction est définie

```
#include <stdio.h>

Int print_table(double, double, double);

void main(void)
{
        int     combien;
        double      end = 100.0;

        combien = print_table(1.0,end,3);
        print_table(end, 200, 15);
}
```

Le compilateur attend des doubles; les conversions sont automatiques

Ici, on ne tient pas compte de la valeur de retour

```
  float calculeFloat(float, float);
  int addition();

void main(void)
{
 int Val ;

 Val = addition();
  printf("val = %d", Val);
}

int addition()
{
 float tmp;
   tmp = calcule(2.5,3) + calcule(5,7.2);
  return (int)tmp;
}

float calcule(float float1, float float2)
{
 return (  (float1 + float2 ) / 2) ;
}
```

Exemple de deux fonctions

CalculeFloat
addition

Règles de visibilité des variables

Le C est un langage structuré en blocs { } , les variables ne peuvent être utilisées que là où elles sont déclarées

```
void    func(int x);
int glo=0;              // variable globale
void    main(void)
{
        int     i = 5, j, k = 2;  //locales à main
        float   f = 2.8, g;

        d = 3.7;
        func (i);
}

void    func(int v)
{
        double d, e = 0.0, f =v; //locales à func

        i++; g--;glo++;
        f = 0.0;
}
```

Le compilateur ne connaît pas d

Le compilateur ne connaît pas i et g

C'est le f local qui est utilisé

Passage des paramètres par valeur

* Quand une fonction est appelée, ses paramètres sont copiés (passage par valeurs)

* La fonction travaille donc sur des copies des paramètres et ne peut donc les modifier

* En C, le passage des paramètres par référence se fait en utilisant des pointeurs (voir plus loin)

* scanf() travaille avec pointeur. C'est le pourquoi du &

Passage des paramètres par valeur

Implémenter l'exemple

```
#include <stdio.h>

void   change(int v);

void   main(void)
{
        int var = 5;

        change(var);

        printf("main: var = %d\n", var);
}

void   change(int v)
{
        v *= 100;
        printf("change: v = %d\n", v);
}
```

```
change: v = 500
main: var = 5
```

FONCTION RENVOYANT UNE VALEUR AU PROGRAMME

```
#include <stdio.h>

int     change(int v);

void   main(void){
        int var = 5;
       int valeur;
        valeur = change(var);

        printf("main: var = %d\n", var);
       printf("main: valeur = %d\n", valeur);
}

int change(int v)
{
        v *= 100;
        printf("change: v = %d\n", v);
       return (v+1);
}
```

Implémenter l'exemple

```
change: v =
main: var =
main: valeur =
```

Exercices

Exercice 1:

- Ecrire une fonction qui convertit le dirham en euro (taux de conversion 10,48).
- Ecrire un programme qui demande à l'utilisateur un nombre réel, le convertit en euro et affiche le résultat en utilisant la fonction précédemment crée.

Exercice 2:

- Ecrire une procédure qui affiche le contenu d'un tableau de taille 5 sous forme {..,…,…,..}
- Ecrire un programme qui saisit un tableau et affiche son contenu en utilisant la procédure précédemment créé.

Une fonction se termine et « rend la main » au code appelant lorsque son exécution rencontre l'instruction : return expression; ou return;

```
#include <stdio.h>

int return_Val(int v);

void   main(void){
        int var = 5;
       int valeur;
        valeur = return_Val(var);

        printf("main: var = %d\n", var);
       printf("main: valeur = %d\n", valeur);
}

int return_Val(int v)
{
        if (v == 10)          return (2*v);
       else                   return (3*v);
}
```

```
return_Val : rien
main: var =
main: valeur =
```

Passage des paramètres par valeur et par adresse

Syntaxe **qui conduit à une erreur:**

```
#include <stdio.h>

void ech(int x,int y)
{
        int tampon;
        tampon = x;
        x = y;
        y = tampon;
}

void main()
{
        int a = 5 , b = 8;
        ech(a,b);
        printf(" a=%d\n ", a) ;
                        printf(" a=%d\n ", a)
        printf(" b=%d\n ", b) ;
}
```

a = ?
b = ?

PASSAGE DES PARAMETRES

PAR VALEUR

Syntaxe **correcte:**

a et b: variables locales à main(). La fonction ech ne peut donc pas modifier leur valeur. On le fait donc en passant par l'adresse de ces variables.

```
h>

nt *y)

void main()
{
 int a = 5 , b = 8 ;
 ech(&a,&b);

 printf(" b=%d\n ", b) ;
}
```

a = ?
b = ?

PASSAGE DES PARAMETRES

PAR ADRESSE

Modification des paramètres

- Si l'on veut qu'une fonction modifie un paramètre, on ne passe pas la variable mais l'adresse de la variable. Il y a copie de l'adresse de la variable. Dans la fonction on va chercher la variable par son adresse.

- Rappels :
  opérateur & : & variable -> adresse de la variable
  opérateur * : * adresse -> valeur qui se trouve à cette adresse

```
int i;
int * adresse_i; /* déclaration d'une adresse d'entier */
i=0;
adresse_i=&i;
printf("%d\n",i); -> 0;
...
```

Appel des fonctions par référence

```
#include <stdio.h>
void    change(int* p);
void    main(void)
{
        int var = 5;
        change(&var);
        printf("main: var = %d\n",var);
}

void    change(int* p)
{
        *p *= 100;
        printf("change: *p = %d\n",*p);
}
```

```
change: *p = 500
main: var = 500
```

Déclarations en "avance"

- <u>Prototype</u>
  En début de programme on donne le type de chaque fonction , son nom, le nombre et les types des arguments
- Information suffisante pour le compilateur.

```
float puiss (float,int);
void main(){
puiss (10.2, 5);
...}
float puiss (float x, int n){
float y=1.0;
if (n>0) for (i=1;i<=n;i++) y = y*x;
else for (i=1;i<=n;i++) y = y/x;
return (y);
}
```

/*Prototype de la fonction puiss*/

/*Appel avant déclaration*/

/*Déclaration de la fonction */

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Exercice:

Refaire le même programme en passant un autre paramètre b à la fonction carre pour y stocker le résultat. Carre ne renvoie plus de retour.

```
void carre3(int a, int b) {
    b = a*a;
}
int main() {
   int n, x=0; printf("n ? ");
   scanf("%d", &n);
   carre3(n, x);
   printf("carre3 = %d\n", x);
   return 0;
}
```

```
//dessiner l'interprétation du code
```